[8 marks] Question 1: Loss Functions

For each of the following loss functions, identify (a) whether the loss can be used for classification, regression, or both; (b) the assumptions, if any, that the loss function makes over the outputs of the model; and (c) one advantage and disadvantage of employing the loss function – e.g., assuming we are operating in the proper regime (classification/regression) for the given loss function, what characteristics of the data/model/learning does the loss function implicitly impose?

[2 marks] Q1.1: Squared Loss (or ℓ_2 loss): $\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}(t^{(i)} - y^{(i)})^2$

[2 marks] Q1.2: Mean Absolute Error (or ℓ_1 loss): $\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}|t^{(i)} - y^{(i)}|$

[2 marks] Q1.3: Binary Cross-Entropy Error: $\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\left[t^{(i)}\log(y^{(i)}) + (1-t^{(i)})\log(1-y^{(i)})\right]$

[2 marks] Q1.4: Huber Loss Error: $h_\delta(y^{(i)}, t^{(i)}) = \begin{cases} \frac{1}{2}(t^{(i)} - y^{(i)})^2 & |t^{(i)} - y^{(i)}| \leq \delta \\ \delta \times (|t^{(i)} - y^{(i)}| - \frac{1}{2}\delta) & \text{otherwise.} \end{cases}$

[20 marks] Question 2: Dealing with troublesome datasets

Datasets are not always neat and tidy. In fact, raw datasets often come with several issues that could potentially harm the validity of the models we build using them. Consider the dataset at this link, where the columns are *index*, x_1, x_2, and *target*, respectively.

[1 mark] Q2.1: Randomly split the dataset into 80% training, 10% validation, and 10% testing. Explain how you make this split reproducible. Use seed 123123 and reuse the resulting partition throughout Q2.

Imbalanced Data: The harm caused by "unclean" datasets may not be immediately obvious when building your models. You could go through the entire process of training, validating, and testing your model without seeing any issues. This might be because you're not evaluating your data under the right lens.

[2 marks] Q2.2: Using the split from Q2.1, report each model's validation accuracy and briefly describe your findings. Use sklearn defaults and the same split throughout Q2. Exclude the index column and treat class 1 as positive.

1. KNN
2. Decision Tree
3. Linear Regression (predict class 1 when the output is ≥ 0.5)[1]
4. Logistic Regression Classifier

[1 mark] Q2.3: While accuracy might be a good way to judge the *overall* performance of the model on the entire dataset, it fails to take into account how the model will perform *for each class*. Why is this a problem?

[1 mark] Q2.4: Explore the dataset and briefly report on the number of classes and the class imbalances. Plot a scatterplot of the dataset to visualize the imbalance.

[1 mark] Q2.5: For each of the four models, report the fraction of true class-0 validation examples correctly classified and the fraction of true class-1 validation examples correctly classified. Compare these per-class results with the overall validation accuracies.

This is where a Confusion Matrix comes into the picture. A Confusion Matrix is a table that compares the model's predicted values against actual values, *per class*. For binary classification, a confusion matrix for your model will generate four values, i.e., True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN). Compute the confusion matrices for all four models on the validation set.

[1 mark] Q2.6: Different combinations of these four values are used as evaluation metrics to provide different insights into the performance of your model. See here for all the possible evaluation metrics. Of these, accuracy has already failed you. It's now time to consider other options. Precision is one such metric for evaluating *how correctly the model has predicted its positive predictions.* The way it is calculated is,

$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{1}$$

Besides precision, recall is another metric to evaluate *how many of the positively labeled instances has the model predicted to be positive*. It is calculated as,

$$\text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{2}$$

[1] In class, labels were encoded as $-1/+1$, with threshold 0. Here they are encoded as 0/1, so we use their midpoint, 0.5.

Precision and Recall are particularly useful in the presence of data imbalances in classes. Why?

[1 mark] Q2.7: The F_1 score combines precision and recall through their harmonic mean. Using the validation set from Q2.1, report the positive-class F_1 score for each of the four models. State how you handle an undefined precision or F_1 score. Reserve the test set for final evaluation after all model and resampling choices are fixed.

[2 marks] Q2.8: Over-sampling: A straightforward solution to imbalanced data is to gather more data. This solution can be pricey, and perhaps infeasible. A workaround to this problem is to randomly duplicate minority-class examples until both classes have equal counts. Over-sample the 80% training set to balance the cardinality per class, build the following models, and report their accuracy, precision, and recall using confusion matrices on the validation set, then compare their performance. Keep the validation set unchanged. You may use sklearn. Explain which model performs better, and briefly explain how you would choose the amount of over-sampling.

1. KNN
2. Decision Tree
3. Linear Regression Classifier
4. Logistic Regression Classifier

[2 marks] Q2.9: Under-sampling: randomly retain only enough majority-class training examples to match the minority-class count. Keep validation and test sets unchanged. Fit the same four models. First report their validation confusion matrices, accuracy, precision, and recall; then compare these results with the original and over-sampled training sets. Discuss the observed trade-offs, the information lost by discarding examples, and how you would choose the amount of under-sampling using validation data.

Outliers: Another issue with the data is that we may have outliers that might lead to problems in classification.

[1 mark] Q2.10: Explore the dataset, and briefly report on which feature(s) have outliers, and what would be a good approach to dealing with them?

[2 marks] Q2.11: Using the original (unresampled) training set, train sklearn KNN models with $k \in \{1, 3, 5\}$ and report their validation confusion matrices. When would outliers not be a problem for KNN? When would they be a problem?

[2 marks] Q2.12: Using the original training set, train sklearn DecisionTreeClassifier models with max_depth=$\{1, 2, 3\}$ and random_state=123123. Report their validation confusion matrices. When would outliers not be a problem for a decision tree? When would they be a problem?

[1 mark] Q2.13: Can either decision trees or KNN be considered uniformly more robust to outliers? Explain how hyperparameters and properties of the data affect the comparison.

[2 marks] Q2.14: Identify two other potential issues with real-world data that require attention before training. Briefly explain why each matters.

[20 marks] Question 3: Bagging and Boosting (20 marks)

Decision trees are considered weak classifiers, making them good candidates for ensemble classifiers. For this part, use the same dataset as in Q2 of this assignment.

[2 marks] Q3.1: Here you will build a simple bagging ensemble classifier consisting of 100 base classifiers. First, perform a 80/10/10 split on your dataset. Then train 100 decision trees as your set of base model (you may use sklearn for the base model using default hyperparameters). Finally, take the maximum vote of the base classifiers as the ensembles final prediction. Plot a histogram showing the accuracy of all 100 base classifiers and the performance of the ensemble classifier on the validation and test sets.

[2 marks] Q3.2: Explore various hyperparameter settings for the base classifiers, and report how this affects the final ensemble performance. Are there tradeoffs to consider?

[1 mark] Q3.3: In (a) and (b), the ensemble's prediction was the maximum vote of the base classifier. Would the results change if instead we averaged the predictions of the base classifiers? If so, how?

[2 marks] Q3.4: What benefits and what drawbacks would you get with using your bagging ensemble classifier over a single decision tree? Would an ensemble model underperform a single classifier?

[1 mark] Q3.5: Random Forest Classifiers are commonly used bagging-based model with decision trees as the base model. On your own, read up on this class of models and describe differences between Random Forests and the simple bagging model you designed in (a).

Bagging ensemble methods make use of the fact that their base models perform well together because their *independence* from each other limits the overlap in each of their weaknesses. Boosting classifiers are another kind of ensemble method that sacrifices the independence of base classifiers to build classifiers that incrementally improve on each other.

[8 marks] Q3.6: Adaptive Boosting (AdaBoost) assigns weights to training examples and trains subsequent classifiers with greater emphasis on previously misclassified examples. Using the same 80% training set as Q3.1, use sklearn's AdaBoostClassifier to build three AdaBoost classifiers, one with each of the following base estimators. Use each classifier's default parameters. Compare their accuracy on the 10% validation set. Comment on any interesting observations.

1. The default AdaBoostClassifier (estimator=None).
2. Decision Tree.
3. Random Forest.

[2 marks] Q3.7: Gradient boosting (GradBoost) is another type of boosting method that tries to use the residual error of previous classifiers to improve subsequent ones. Read up on how GradBoost and AdaBoost compare with each other and identify and explain one difference between how they work.

[2 marks] Q3.8: XGBoost is a popular ensemble classifier that uses eXtreme Gradient Boosting, i.e., scalable distributed gradient boosting through parallel boosting. Using the XGBoost python library, compare extreme gradient boosting to a GradBoostClassifier, and AdaBoostClassifier. Use the XGBoost library and the default implementation of all three for this comparison.